

I/O Programming in Java

Stream, Byte Stream, Character Stream, Readers and Writers

These notes are designed for **beginner-level Core Java students** and can be directly used for classroom teaching.

1. Introduction to I/O Programming

I/O stands for **Input/Output**.

In Java, I/O programming is used when a program needs to **receive data from a source** or **send data to a destination**.

Examples

- Reading data from keyboard
- Displaying output on screen
- Reading a text file
- Writing data into a file
- Reading an image
- Copying a file
- Sending or receiving data through a network

Simple idea

```
Input
  ↓
Java Program
  ↓
Output
```

For example:

```
Keyboard → Java Program → Screen
```

Or:

```
File → Java Program → File
```

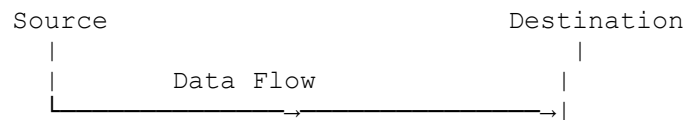
Java provides many classes for performing these operations, mainly through the **java.io** package.

```
import java.io.*;
```

2. What is a Stream?

A **stream** is a flow of data between a program and a source/destination.

Think of a stream like a **water pipe**.

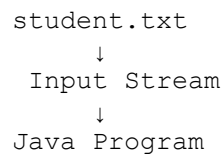


In Java:

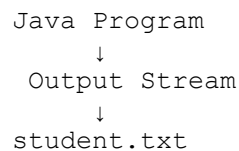
- **Input Stream** → data comes **into** the program.
- **Output Stream** → data goes **out of** the program.

Example

Suppose we read a file:



Suppose we write into a file:



Important point

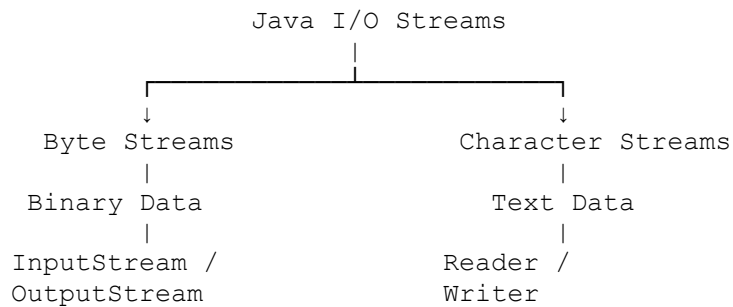
A stream does not necessarily mean a file.

Streams can be used with:

- Files
- Keyboard
- Memory
- Network connections
- Other data sources

3. Types of Streams in Java

Java mainly provides two important types of streams:



Byte Streams

Used mainly for **binary data**.

Examples:

- Images
- Audio
- Video
- PDF
- Binary files

Main classes:

InputStream
OutputStream

Character Streams

Used mainly for **text data**.

Examples:

- .txt
- .java
- .csv
- .html

Main classes:

Reader
Writer

4. Byte Stream

A **byte stream** handles data in the form of **8-bit bytes**.

It is useful when working with binary data.

Java provides two main abstract classes:

```
InputStream  
OutputStream
```

InputStream

Used to **read bytes**.

```
File  
↓  
InputStream  
↓  
Program
```

OutputStream

Used to **write bytes**.

```
Program  
↓  
OutputStream  
↓  
File
```

5. Important Byte Stream Classes

Some commonly used byte-stream classes are:

Class	Purpose
InputStream	Reads bytes
OutputStream	Writes bytes
FileInputStream	Reads bytes from a file
FileOutputStream	Writes bytes to a file
BufferedInputStream	Reads bytes efficiently using buffering
BufferedOutputStream	Writes bytes efficiently using buffering

For beginners, the most important ones are:

```
FileInputStream  
FileOutputStream
```

6. FileInputStream

`FileInputStream` is used to **read data from a file byte by byte**.

Example 1: Reading a file

Create a file named:

`data.txt`

Put this inside it:

Hello Java

Java program:

```
import java.io.FileInputStream;
import java.io.IOException;

public class ReadFile {
    public static void main(String[] args) throws IOException {

        FileInputStream input = new FileInputStream("data.txt");

        int data;

        while ((data = input.read()) != -1) {
            System.out.print((char) data);
        }

        input.close();
    }
}
```

Output

Hello Java

Explanation

```
FileInputStream input =
    new FileInputStream("data.txt");
```

This opens the file for reading.

```
input.read()
```

Reads one byte from the file.

```
(char) data
```

Converts the byte into a character for displaying text.

```
input.close();
```

Closes the file.

Flow

```
data.txt  
  ↓  
FileInputStream  
  ↓  
read()  
  ↓  
Java Program  
  ↓  
Screen
```

7. FileOutputStream

`FileOutputStream` is used to **write bytes into a file**.

Example 2: Writing to a file

```
import java.io.FileOutputStream;
import java.io.IOException;

public class WriteFile {
    public static void main(String[] args) throws IOException {

        FileOutputStream output =
            new FileOutputStream("data.txt");

        String message = "Welcome to Java I/O";

        output.write(message.getBytes());

        output.close();

        System.out.println("Data written successfully.");
    }
}
```

The program creates `data.txt` if it does not already exist.

The file will contain:

```
Welcome to Java I/O
```

Important method

```
output.write()
```

writes data to the file.

Since `FileOutputStream` works with bytes, we use:

```
message.getBytes()
```

to convert the `String` into bytes.

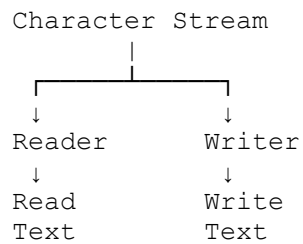
8. Character Stream

Byte streams work with **bytes**, while character streams are designed specifically for **text/character data**.

Character streams use:

Reader
Writer

Basic structure

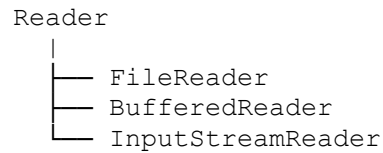


Character streams are particularly useful when working with text because they handle characters rather than individual raw bytes.

9. Reader

`Reader` is an **abstract class** used for reading character data.

Some important subclasses are:



FileReader

`FileReader` is used to read text files.

Example 3: Using FileReader

Create:

`student.txt`

Content:

Welcome Students

Program:

```
import java.io.FileReader;
import java.io.IOException;

public class ReadText {
    public static void main(String[] args) throws IOException {

        FileReader reader = new FileReader("student.txt");

        int ch;

        while ((ch = reader.read()) != -1) {
            System.out.print((char) ch);
        }

        reader.close();
    }
}
```

Output

Welcome Students

Explanation

```
FileReader reader =  
    new FileReader("student.txt");
```

Opens the text file.

```
reader.read()
```

Reads one character at a time.

```
(char) ch
```

converts the value into a character.

```
reader.close();
```

closes the reader.

10. Writer

`Writer` is an **abstract class** used for writing character data.

Important subclasses include:

```
Writer
├── FileWriter
├── BufferedWriter
└── OutputStreamWriter
```

FileWriter

`FileWriter` is commonly used to write text into a file.

Example 4: Using FileWriter

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteText {
    public static void main(String[] args) throws IOException {

        FileWriter writer =
            new FileWriter("student.txt");

        writer.write("Hello Students!");

        writer.close();

        System.out.println("Data written successfully.");
    }
}
```

File content

Hello Students!

11. Reader vs Writer

Reader	Writer
Used for reading text	Used for writing text
Input operation	Output operation
Reads characters	Writes characters
Main class for character input	Main class for character output
Example: <code>FileReader</code>	Example: <code>FileWriter</code>

Simple way to remember:

Reader → Reads
Writer → Writes

12. Byte Stream vs Character Stream

Feature	Byte Stream	Character Stream
Main purpose	Handles byte data	Handles character/text data
Unit	Byte	Character
Main classes	<code>InputStream</code> , <code>OutputStream</code>	<code>Reader</code> , <code>Writer</code>
Common file classes	<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>
Suitable for	Binary data	Text data
Examples	Image, audio, video	TXT, Java, CSV
Encoding handling	More low-level	Designed for character data

Easy memory trick

BYTE → Binary
CHARACTER → Text

13. File Copy Using Byte Stream

One useful application of byte streams is **copying files**.

Example 5: Copy a file

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopyFile {
    public static void main(String[] args) throws IOException {

        FileInputStream input =
            new FileInputStream("source.txt");

        FileOutputStream output =
            new FileOutputStream("copy.txt");

        int data;

        while ((data = input.read()) != -1) {
            output.write(data);
        }

        input.close();
        output.close();

        System.out.println("File copied.");
    }
}
```

Flow

```
source.txt
  ↓
FileInputStream
  ↓
read()
  ↓
Java Program
  ↓
write()
  ↓
FileOutputStream
  ↓
copy.txt
```

This approach can also be used for binary files such as images because byte streams preserve the raw byte data.

14. Buffered Streams

Reading or writing one byte/character at a time can be inefficient for large files.

Java provides **buffered streams**.

A buffer is a temporary area of memory used to store data while reading or writing.

```
File
↓
Buffer
↓
Program
```

Instead of repeatedly communicating with the file, Java can work with a block of data in memory.

Important classes

```
BufferedInputStream
BufferedOutputStream
BufferedReader
BufferedWriter
```

15. BufferedReader

`BufferedReader` is commonly used to read text efficiently.

One useful method is:

```
readLine()
```

It reads a complete line of text.

Example 6: Reading a line

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadLines {
    public static void main(String[] args) throws IOException {

        BufferedReader br =
            new BufferedReader(new FileReader("student.txt"));

        String line;
```

```
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }

        br.close();
    }
}
```

Suppose `student.txt` contains:

```
Rahul
Amit
Neha
Priya
```

Output:

```
Rahul
Amit
Neha
Priya
```

Why use `BufferedReader`?

Instead of reading one character at a time:

```
reader.read()
```

we can read a complete line:

```
br.readLine()
```

This is convenient for text files.

16. `BufferedWriter`

`BufferedWriter` is used to write text efficiently.

Example 7

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class WriteLines {
    public static void main(String[] args) throws IOException {

        BufferedWriter bw =
            new BufferedWriter(new FileWriter("student.txt"));

        bw.write("Java Programming");
        bw.newLine();
    }
}
```

```

        bw.write("Data Science");
        bw.newLine();
        bw.write("Artificial Intelligence");

        bw.close();

        System.out.println("Data written.");
    }
}

```

File content:

```

Java Programming
Data Science
Artificial Intelligence

```

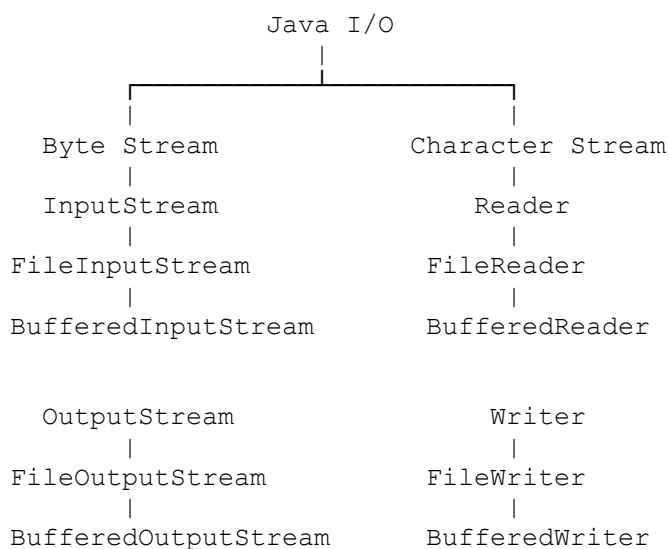
Important method

```
bw.newLine();
```

moves the cursor to a new line.

17. Input and Output Stream Hierarchy

A simplified hierarchy can be remembered as:



Students do not need to memorize every class initially. Understand the **purpose and relationship** first.

18. Reading from Keyboard

Java programs can also take input from the keyboard.

One common approach is using `Scanner`.

```
import java.util.Scanner;

public class KeyboardInput {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Hello " + name);

        sc.close();
    }
}
```

Output

```
Enter your name: Ishan
Hello Ishan
```

Here:

```
Keyboard
  ↓
System.in
  ↓
Scanner
  ↓
Java Program
```

19. System.in and System.out

Java provides three standard streams:

```
System.in
System.out
System.err
```

System.in

Used for standard input.

Usually:

Keyboard → `System.in`

System.out

Used for normal output.

```
System.out.println("Hello");
```

System.err

Used for error messages.

```
System.err.println("Error occurred");
```

Simple diagram

```
Keyboard
  ↓
System.in
  ↓
Program
  ↓
System.out
  ↓
Screen
```

20. Why Do We Need `close()` ?

When a file is opened, the program uses system resources.

Therefore, after completing the operation, we should close the stream.

Example:

```
FileReader reader = new FileReader("data.txt");

// Reading...

reader.close();
```

Why close?

- Releases system resources
- Prevents unnecessary resource usage
- Helps ensure output data is properly written
- Good programming practice

Remember:

Open → Use → Close

21. Exception Handling in I/O

I/O operations can produce errors.

For example:

- File does not exist
- File cannot be accessed
- Permission denied
- Disk problem
- Invalid input/output operation

Therefore, Java requires appropriate exception handling for many I/O operations.

For beginners, we can use:

`throws IOException`

Example:

```
import java.io.FileReader;
import java.io.IOException;

public class Demo {
    public static void main(String[] args)
        throws IOException {

        FileReader reader =
            new FileReader("data.txt");

        reader.close();
    }
}
```

`IOException` means an **Input/Output related problem** has occurred.

22. Try-Catch Approach

Instead of `throws`, we can handle the exception using `try-catch`.

```
import java.io.FileReader;
import java.io.IOException;

public class Demo {
    public static void main(String[] args) {

        try {
            FileReader reader =
                new FileReader("data.txt");

            System.out.println("File opened.");

            reader.close();
        }
    }
}
```

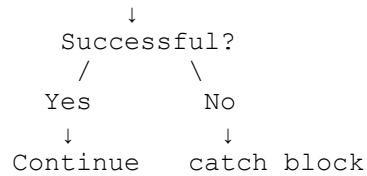
```

        } catch (IOException e) {
            System.out.println("File error occurred.");
        }
    }
}

```

Flow

Try to open file



23. Quick Comparison of Important Classes

Class	Used For
<code>InputStream</code>	Reading bytes
<code>OutputStream</code>	Writing bytes
<code>FileInputStream</code>	Reading file bytes
<code>FileOutputStream</code>	Writing file bytes
<code>Reader</code>	Reading characters
<code>Writer</code>	Writing characters
<code>FileReader</code>	Reading text files
<code>FileWriter</code>	Writing text files
<code>BufferedReader</code>	Efficient text reading
<code>BufferedWriter</code>	Efficient text writing

24. Real-Life Analogy

Imagine a water pipeline.

Input Stream

Water comes **towards your house**.

Water Source → Pipe → House

Similarly:

File → `InputStream` → Program

Output Stream

Water goes **from your house** somewhere else.

House → Pipe → Destination

Similarly:

Program → OutputStream → File

Reader/Writer

Imagine the data is a **written message**.

Reader → Reads the message

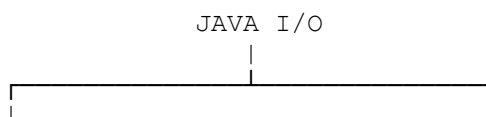
Writer → Writes the message

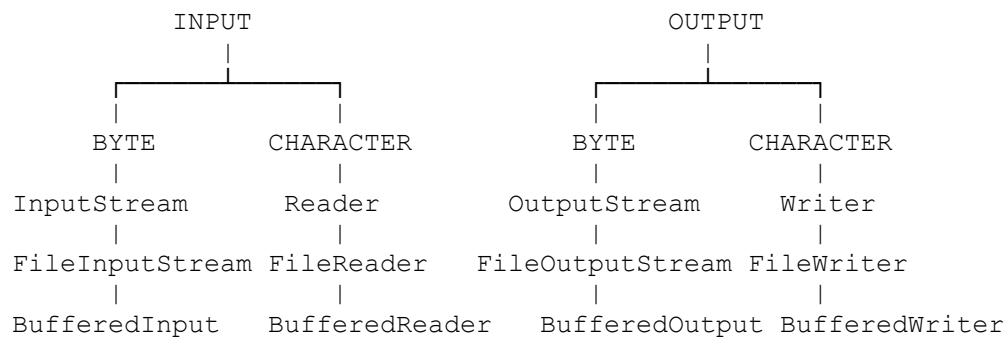
25. Important Points to Remember

1. **I/O** means Input/Output.
 2. A **stream** represents a flow of data.
 3. **Input** brings data into the program.
 4. **Output** sends data from the program.
 5. **Byte streams** work with bytes.
 6. Byte streams are useful for **binary data**.
 7. `InputStream` is used for byte input.
 8. `OutputStream` is used for byte output.
 9. `FileInputStream` reads bytes from a file.
 10. `FileOutputStream` writes bytes to a file.
 11. **Character streams** are designed for text.
 12. `Reader` is used to read character data.
 13. `Writer` is used to write character data.
 14. `FileReader` reads text files.
 15. `FileWriter` writes text files.
 16. `BufferedReader` can read text line by line.
 17. `BufferedWriter` can write text efficiently.
 18. Streams should generally be **closed after use**.
 19. I/O operations may generate `IOException`.
 20. `System.in` is standard input and `System.out` is standard output.
-

26. Overall Concept

The complete concept can be summarized as:





One-line memory trick

Byte Stream → Binary Data

Character Stream → Text Data

Input → Read

Output → Write

Reader → Read Characters

Writer → Write Characters

This gives students the foundation needed before moving to topics such as **Buffered Streams, Data Streams, Object Serialization, RandomAccessFile, and Java NIO.**